

Week 9 - Spatial Data Worksheet

Hannah Wauchope

2023-11-13

This worksheet is a fun bunch of exercises to introduce you to the world of working with spatial data in R. We're going to answer a few simple questions, just to give you an idea of what's possible and what's out there. This worksheet barely scratches the surface of what you're able to do with spatial data, but hopefully it will at least give you the mental framework to know how/where to look if you decided to do an analysis or make a map of your own.

We're going to be doing a few simple exercises that consider where we have recorded data of three species of sloth in South America. We'll look at the average climates the sloths live in (using raster data), and plot sloth information by country (using vector data).

And if you're feeling adventurous, we finish with me issuing you an (optional) challenge: can you compare the average human footprint of sloth individuals that occur inside vs outside protected areas?

Part 1 - Getting our sloth data

First things first, let's download some data on occurrence records of sloths in South America. We're going to get this data from GBIF, the Global Biodiversity Information Facility.

Exercise 1

Go to gbif.org, click on "occurrences" (above the search bar), then "Map". Note how many occurrences there are with coordinates (shown at the top of the screen!!), try and have a scroll through the map. You'll note it's probably very laggy as it's trying to load so much data. Switch to gallery and have a scroll. Basically - just appreciate the insane amount of data on offer here.

Exercise 2

(note - code provided for this exercise)

Okay, let's get our sloth data. We're using the GBIF API which is accessed via the `rgbif` package. Now, if you want to make big downloads from `rgbif`, you need to create a `gbif` account, and use different calls to the ones we use today. See here (<https://docs.ropensci.org/rgbif/>) for more info. But we're focusing on a few fairly rare sloth species, so we can access the data without an account.

To access data on a species (or any taxonomic group), you need to know its 'key', a number GBIF assigns it that is unique to that species (or genes, family, class etc). The `name_backbone` function gives us all the keys for the pale toed sloth. Look at the output when you run that first line. We could use the numbers given in the "key" columns to get data on the genus, family, class etc of this sloth. As it is we want the species, which means we want the usage key.

Now run the next three lines. These take the usage key from three sloth species, and get occurrence data. Take a look at the output.

Finally, we bring that data together. Now we have coordinates on sloth occurrences!

Exercise 3

Now to get spatial. Let's turn this tibble into a spatial object.

Think about what you've learnt from the powerpoint - is individual coordinate data like this likely to become a raster or vector spatial object?

Now, **run the `vect()` function from terra to create your object, which I want you to name "SlothPts"**. Run `?vect` to see instructions for how the function works. You will need to specify the dataset you're building the vector from ("Sloths"), plus two more calls in the function, `geom` and `crs`:

`geom` tells the function which columns in your data set are your coordinates. E.g. if you had columns in your dataset called "lat" and "long" you would type `geom=c("long", "lat")`. What will you add to your function? (Hint - look at the column names in `sloths` !) IMPORTANT: longitude always goes first! this is cos R thinks of it as x and y, and longitude represents the horizontal or x axis of coordinates, and latitude the y axis.

`crs` stands for Coordinate Reference System. You'll note at the start of this code file I've already defined for you the "Standard" crs, WGS 1984 as WGS84. Because our lat longs are in "normal" coordinates, we'll specify this crs. This means you'll add "`crs=WGS84`" to your function.

Now, to check things are looking somewhat right, try running `plot(SlothPts)` - you should see a plot with a bunch of points scattered about

Part 2 - Get some climate data

Let's say we now want to understand what kind of climates our sloths tend to live in. We are going to find out what the average temperature is at each sloth occurrence point. To do this we'll use raster data of climate.

First, head over to worldclim.org and have a click around. This is a commonly used resource that provides current and future global raster data of climates. You can download data by clicking on the website, but instead we'll use the package `geodata` which allows you to download directly from within R.

Exercise 4

Let's download some climate data using the package `geodata`. Run `?geodata` to see what kinds of datasets it offers. We're going to download `worldclim` global data - find and click on the relevant function to see what you should code.

Create an object called "Temp" using this function. We would like the `var` "tavg" (i.e. average temperature) data at a `res` (resolution) of 10min (this is a very coarse resolution, but otherwise the data takes too long to download). The path we will use is `tempdir()`, which just saves the file somewhere temporary. If you were doing this analysis more comprehensively, you could save your file to a useful filepath so you can access it from there later (especially useful if downloading large files).

Exercise 5

Run `Temp` in the console to inspect your raster. Have a look at what you get for a summary: the dimensions tell you what how many cells are in the raster (rows by columns) and how many layers there are, resolution says how large each grid cell is, and the extent gives you the coordinate space that the raster covers.

You may be wondering what it means by 12 layers. Run `plot(Temp)` to see if this helps you understand.

There's 12 layers - this is one layer for every month of the year. So layer 1 gives average temperature in January, layer 2 February etc etc.

Working with temperatures for every month will be a bit of a faff for today, let's just get the overall annual average temperature. Maths with rasters is blessedly easy, run:

```
Temp <- mean(Temp)
plot(Temp)
```

To see what you get. When you have rasters of the same resolution and extent, simple maths like this is easy. It just takes the average in every grid cell across the layers! Other maths functions also work, we could calculate the mean, sum, standard deviation etc like this.

A quick final thing, run the following

```
names(Temp) <- "MeanTemp"
```

Now the values in the raster have a more useful heading (before it was just 'mean')

Part 3 - Cropping and Projecting Data

Okay. Now we have occurrence points of sloths as a point vector, and a raster of climate data. Sloths only occur in South America, so it makes sense to crop our data, and also put them in a better projection for South America. Cropping is often easiest when done with a latitude/longitude based projection (like WGS 84) because it's easy to think in coordinates, so we'll do that before we change projection.

Exercise 6

To crop spatial data, you first need to create an "Extent" object, that is basically a rectangle that defines the area you would like to crop to. You use the call `ext()` and within you specify four coordinates: `xmin`, `xmax`, `ymin`, `ymax`. `x` corresponds to longitude, and `y` to latitude. For South America, these coordinates are: 85.0, -30.0, -60.0, 15.

(Nb: Latitudes South of the equator are coded as negative, as are longitudes West of the central meridian. So South America extends from 85° West to 30° West, and 60° South to 15° North)

Now, create an object called **SAExt** using `ext()`

Exercise 7

Cropping is very easy now. You just use the function `crop`, and first specify the object you want to crop (in our case either `Temp` or `SlotPts`), and then the extent object you want to crop it to. In other words it's `crop(VectorOrRaster, Ext)` **Crop Temp and SlothPts to the South America extent**, giving them the same names as before as we don't have any desire to keep the data outside South America.

Now, check your cropping has worked. First run `plot(Temp)` and check your raster now shows South America. Then run `points(SlothPts)`, and check the points overlay on top. If anything looks awry, check that you have your coordinates in the right order from when you created your `SAExt` object!

Plotting like this isn't useful for making pretty publication plots, but it's very handy for checking things are working as they should.

Exercise 8

Lovely, now it's time to reproject our data. WGS 84 projection is the general baseline projection, but it's bad practice to do raster calculations with it. Why? Because raster gridcells change in size depending on where they are in the world, as they're based on lat/longs. E.g. At the equator there is roughly 111km

between each degree of longitude. At 40deg North, there is only 85km between each degree, and if you're close to the north or south pole then the distance could be a few metres, or less! Instead, it's good practice to work in equal area projections, where gridcells are the same size and measured in metres. If you're analysing the whole world, the "Mollweide" projection is a good one to use.

In this case, we're going to use the South America Albers Equal Area Projection, which you can find info on here: <https://epsg.io/102033> (<https://epsg.io/102033>). This website is great for providing info on all the billions of different map projections available out there. The code we need from this website is the "PROJ.4" string of code. Scroll down to the "Export" section of the website, and copy the PROJ.4 code. It should look similar in structure to the WGS CRS code at the start of the script.

Create a new object, underneath the WGS CRS line of code (Line 20) that defines a new crs using this string, name it "SACRS". (It's good practice to define all your CRS's at the start of the script)

Exercise 9

And now, it's very easy to reproject our data. Use the function `project()`, in which you first specify what you want to project (either your Temp or SlothPts objects), and then you specify the new CRS (in this case SACRS). In other words, it's `project(VectorOrRaster, CRS)`

First, **reproject the Temp file using this method**. Make a new object this time, called TempSA

Now, run

```
plot(TempSA)
points(SlothPts)
```

What do you notice? Why has this happened?

Now, reproject SlothPts, naming it SlothPtsSA, and then run

```
plot(TempSA)
points(SlothPtsSA)
```

Much better.

Part 4 - Extracting data

Exercise 10

Okay, now let's find out what the temperature is for each of our sloth occurrence points! To do this, we use the `extract()` function. Run `?extract` to understand the call, we will input our raster as the x object, i.e. the object that provides the information, and our points vector as the y object, i.e. the object that says at which points we want the information. Also, specify that "bind = TRUE" as then it will return a the full vector including all the columns in SlothPtsSA. In other words, it's

`extract(Raster, Points, bind=TRUE)`. *Run the extract function and name your new object SlothPtsSA also*, as essentially this function will just add a column for temperature.

Run `head(SlothPtsSA)` and look at what you've created. The new column is from the raster, and shows mean temperature at each point!

Exercise 11

Create a quick ggplot that shows a boxplot of mean temperatures for each of the sloth species. You'll need to include SlothPtsSA as `as.data.frame(SlothPtsSA)` as ggplot doesn't understand how to get data out of vector objects. Reminder that the boxplot call is `geom_boxplot` and you need to specify what you want on your x axis (species) and y axis (mean temp) within `aes()`

Part 5 - Working with Polygon Vectors

Let's say we want to make a map of South America, and we want to colour each country by how many sloth occurrence points it has. How might we do this?

Exercise 12

First, we need a polygon vector of country boundaries. We'll also get this from geodata. Run `?geodata` and find and click on the function call that will give us boundaries for the *world's* countries. **Create an object called CountryPolys**, specifying that we want the lowest level of resolution (5) (otherwise it's a slow download), and as before, make `path=tempdir()` (unless you want to save this file in which case, whatever file path you like).

Run `plot(CountryPolys)` and `head(CountryPolys)` to look at what information is included in this vector file.

Exercise 13

Now, let's crop this also to just South America. Do this exactly as you cropped the temp raster and the sloth points vector (exercise 7). Keep the object name the same.

Exercise 14

Now run `plot(CountryPolys)` and take a look. To me, this projection actually looks pretty alright, so let's just keep it as WGS 84! We're not doing any analyses that mean we need an equal area projection. Think about what this means for which sloth points object we'll be using in the next exercise, where we'll overlay the points on these country polygons. Before proceeding, run `plot(CountryPolys)` then `points(WhateverSlothPtsObjectYou'veDecidedToUse)` to check everything's looking right.

Part 6 - Extracting data from polygons

Exercise 15

Now we want to find out which country each point falls within. Happily, `extract` is here to help us again! We run it just like in Exercise 10, except this time don't include the `bind=TRUE` part as this doesn't work if both x and y are vectors (I don't know why... it should). This annoyingly means we need to create a new object for this extract, call it "SlothExtract". Remember that with `extract` you put the information layer first (CountryPolys) and the points layer second (SlothPts). So `extract(Polygon, Points)`

Once you've got your object, double check you've done it right by comparing `nrow(SlothPts)` with `nrow(SlothExtract)`. They should be the same.

Now, run:

```
SlothPts$NAME_0 <- SlothExtract$NAME_0
head(SlothPts)
```

You'll see that now we've added countries in as a column to the SlothPts file. We've kept the awkward name "NAME_0" to make the next next step easier.

Exercise 16

Now for a quick bit of good old data wrangling. **Can you calculate how many sloth points are within each country?** (Let's just go overall rather than by species, to keep things simple). To help you out, here's the code you'll need, you just need to replace "???" with the right things.

```
SlothCountries <- as.data.frame(???) %>%  
  group_by(???) %>%  
  count()
```

Note that we start with `as.data.frame()`, as pipes/tidyverse don't understand how to look at the data within vector objects (well, at least not yet, I think people are working on it).

Name your summarised object SlothCountries

Exercise 17

Nearly there! Finally, we want to add this information *back* into the CountryPolys vector, to enable us to map it. Now, if we were working with data frames in the tidyverse we would use `left_join`, but tidyverse doesn't understand spatial data. Instead, the terra package has adapted the 'merge' function from Base R, which works in the same way.

```
CountryPolys <- merge(CountryPolys, SlothCountries, by="NAME_0", all.x=T)
```

This says the merge the information in SlothCountries into the CountryPolys data, matching by Country Name, and keeping all the rows in the CountryPolys dataset.

Part 7 - Plotting spatial objects

Exercise 18

And now it's time to map! We use the ggplot package tidyterra to help with this. The ggplot call to plot a vector object (whether points, polygons or lines) is `geom_spatvector()`. You don't need to specify x and y in the aes, it just knows how to read it. But, for our purposes, you should specify `fill=n` (or whatever else you've named your column that gives the number of sloths occurrences in each country!) within the `aes()`, so that each country is coloured by the number of sloth occurrences.

Note: for some reason ggplot gets a bit stupid with this spatial data, and you need to explicitly say "data=" before you specify data anywhere.

Try adding the occurrence points over the top, coloured by species! (hints: the points are also a spatvector, and you should say that colour should vary by species within `aes()`)

A NOTE: You'll notice that this is quite a different approach to mapping as taught in coding club, or in your Challenge 1. There are ways to access base maps, as you've been shown, but these are very difficult to manipulate or adjust as you'd like. It's personal preference, but I much prefer downloading my own vector files like we've done here, as you have much more flexibility. Also, I find it *much* easier to project your data into whatever projection you want to use *before* you make your ggplot, it just works better that way.

A bonus thing

The extract function is super powerful. Say you wanted to know what the average temperature was in each south american country? All you'd need to do is to run extract with the temp raster and the country polygons!

```
CountryPolysSA <- project(CountryPolys, SACRS) #Gotta make sure they're in the same
projection

plot(TempSA) #Rough plot to check everything's looking right
plot(CountryPolysSA, add=TRUE)

SAClim <- extract(TempSA, CountryPolysSA, fun=mean, bind=TRUE) #Now, this extract w
ill take all the raster cells within each country polygon, and take the mean! Handy
.

head(SAClim) #Take a look
```

And now, an optional challenge!

If you've finished this and are keen to test your skills, here's my challenge for you:

In this project folder, there data folder called "WDPA". This contains a shapefile of protected area polygons in the country of Suriname (named "SurWDPA.shp").

I want to know: **For the country of Suriname** does the human footprint value differ between protected and unprotected sloth occurrence points? Write a workflow and see if you can make this happen! (Hint: check `?geodata` to find the `footprint` function which will tell you more about the human footprint data)

Create a simple boxplot that compares human footprint values for points inside and outside protected areas. The first person to show or send me this boxplot wins a prize!

(Double triple prize if you can then make a map of the human footprint raster with protected areas and points overlayed. As a hint, you'll need the function `mask` to get your raster looking right)